

9 Laboratorinis darbas. AES

- 1977m. IBM kartu su NSA (*National Security Agency*) sukūrė DES (*Data Encryption Standard*). DES tapo populiariausiu simetriniu šifru.
- 1997m. DES buvo pirmą kartą nulaužtas.
- 1997m. NIST (*National Institute of Standards and Technology*) paskelbė konkursą rasti algoritmui atitinkantį pažangųjį šifravimo standartą (*Advanced Encryption Standard* arba AES).
- Konkurse dalyvavo 15 algoritmų. Konkursą laimėjo *Rjindael* algoritmas, kurį pasiūlė Vincent Rijmen ir Joan Daemen.
- AES yra vienintelis viešas šifravimo algoritmas, kurį naudoja NSA.

Blokiniai šifrai

AES, kaip ir visus blokinius šifrų, galima aprašyti šia procedūra:

- 1 Atviro teksto / šifro skaidymas į blokus
- 2 Blokų šifravimas / dešifravimas raktu
- 3 Blokų apjungimas

Blokai

AES šifras dirba su atviru tekstu suskaidytu į 128 bitų blokus (16 baitų). Tiek atviro teksto, tiek rakto baitai išdėstomi į 4x4 matricą stulpelių pirmumo principu. Atviro teksto matricos vadinamos būsenomis (angl. *state*)

$$\underbrace{\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}}_{\text{būsena}} \quad \underbrace{\begin{bmatrix} k_0 & k_4 & k_8 & k_{12} \\ k_1 & k_5 & k_9 & k_{13} \\ k_2 & k_6 & k_{10} & k_{14} \\ k_3 & k_7 & k_{11} & k_{15} \end{bmatrix}}_{\text{raktas}}$$

Teksto konvertavimas į būseną

Tarkime turime tekstą *Tyli kiaulė gilią šaknį knisa*. Konvertavus tekstą į baitus UTF-8 koduote, gauname: 54796c69206b6961756cc4972067696c69c48520c5a1616b6ec4af206b6e697361. Šių simbolių seką sudaro 33 baitai, todėl dalinama į tris blokus. Paskutiniame bloke pridėdami nuliai.

$$\begin{bmatrix} 54 & 20 & 75 & 20 \\ 79 & 6b & 6c & 67 \\ 6c & 69 & c4 & 69 \\ 69 & 61 & 97 & 6c \end{bmatrix} \quad \begin{bmatrix} 69 & c5 & 6e & 6b \\ c4 & a1 & c4 & 6e \\ 85 & 61 & af & 69 \\ 20 & 6b & 20 & 73 \end{bmatrix} \quad \begin{bmatrix} 61 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Šifravimas ir dešifravimas

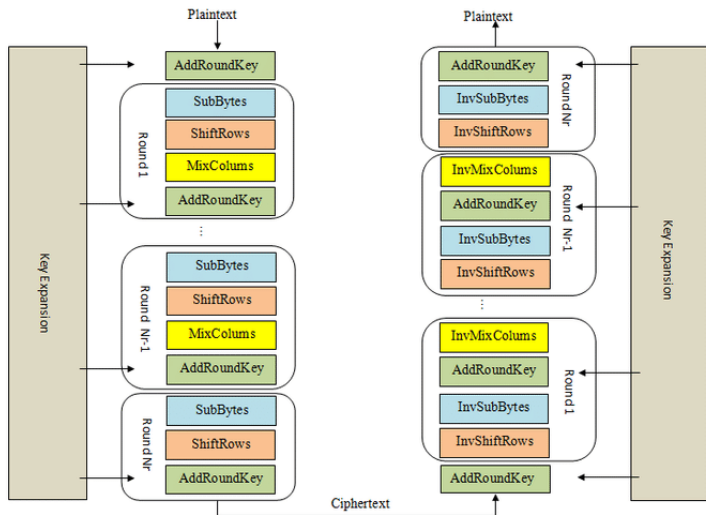
Šifravimas ir dešifravimas yra vykdomas raundais. Raundams atlikti reikalingos 4 procedūros ir joms atvirkštinės. Kiekviena procedūra atlieka užduotį - sukeisti arba sumaišyti baitus. Procedūros:

- AddRoundKey
- SubBytes ir InvSubBytes
- ShiftRows ir InvShiftRows
- MixColumns ir InvMixColumns

Raktai

AddRoundKey procedūroje prie bloko pridedamas raktas. Kiekvieną kartą vykdant AddRoundKey procedūrą, yra pridedamas vis kitas raktas, taigi, jei raundų yra N šifravimui reikalingi $N + 1$ raktų bloko dydžio raktų. Vietos taupymo tikslais AES naudoja KeyExpansion procedūrą leidžiančią sugeneruoti raktų srautą.

Šifravimas ir dešifravimas



Tarkime šifravimą sudaro N raundų. Šifravimui bus reikalinga $N + 1$ raktų r .

➊ Prieš pirmą raundą prie bloko pridedamas pirmas raktas r_1 AddRoundKey procedūra
 $1 \leq n < N$ raundams su bloku atliekamos procedūros tokia tvarka:

- ➊ SubBytes
- ➋ ShiftRows
- ➌ MixColumns
- ➍ AddRoundKey su r_{n+1}

N -ajame raunde MixColumns procedūra praleidžiama:

- ➊ SubBytes
- ➋ ShiftRows
- ➌ AddRoundKey su r_{N+1}

Dešifravimui atliekami N raundų, kuriuose visos funkcijos atliekamos atvirkščia tvarka. Naudojami tie patys raktai r .

- 1 Prieš pirmą raundą prie bloko pridedamas paskutinis raktas r_{N+1} AddRoundKey procedūra

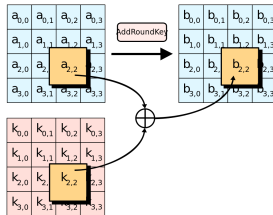
$1 \leq n < N$ raundams su bloku atliekamos procedūros tokia tvarka:

- 1 InvShiftRows
- 2 InvSubBytes
- 3 AddRoundKey su r_{N+1-n}
- 4 InvMixColumns

N -ajame raunde InvMixColumns procedūra praleidžiama:

- 1 InvShiftRows
- 2 InvSubBytes
- 3 AddRoundKey su r_1

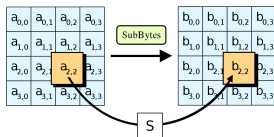
AddRoundKey



AddRoundKey procedūroje prie kiekvieno būsenos baid pridamos atitinkamas rakto baidas XOR operacija (žymima $\oplus$). Pvz. $54_{16} \oplus 62_{16} = \oplus \frac{01010100_2}{01100010_2} = 00110110 = 36_{16}$. Šios operacijos tikslas - pakeisti baidus.

Kadangi XOR operacija yra atgręžiamo, AddRoundKey operacijos atvirkštinė operacija yra AddRoundKey.

$$\underbrace{\begin{bmatrix} 54 & 20 & 75 & 20 \\ 79 & 6b & 6c & 67 \\ 6c & 69 & c4 & 69 \\ 69 & 61 & 97 & 6c \end{bmatrix}}_{\text{būsena}} \oplus \underbrace{\begin{bmatrix} 62 & 30 & 35 & 34 \\ 35 & 33 & 33 & 39 \\ 32 & 32 & 62 & 38 \\ 34 & 38 & 34 & 30 \end{bmatrix}}_{\text{raktas}} = \underbrace{\begin{bmatrix} 36 & 10 & 40 & 14 \\ 4c & 58 & 5f & 5e \\ 5e & 5b & a6 & 51 \\ 5d & 59 & a3 & 5c \end{bmatrix}}_{\text{rezultatas}}$$



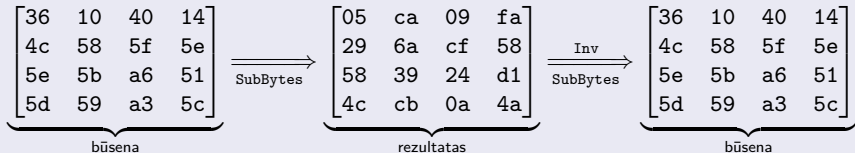
00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f	00	01	02	03	04	05	06	07	08	09	0a	0b	0c	0d	0e	0f
00	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76	52	09	6a	d5	30	36	a5	38	b1	40	a3	9e	81	f3	d7
10	ca	82	c9	7d	fa	59	47	10	ad	d4	a2	af	5c	a4	72	c8	7c	e3	39	02	9b	2f	ff	87	34	ae	43	44	c4	de	
20	b7	fd	93	26	36	3f	1f	cc	34	a5	a5	f1	71	d8	31	15	54	7b	94	32	a6	c2	23	3d	ee	4c	95	0b	42	fa	
30	04	c7	23	c3	18	96	05	5a	07	12	80	a2	ab	27	b2	75	08	2e	a1	66	28	d9	24	b2	76	5b	a2	49	6d	8b	
40	59	83	2c	1a	1b	6e	5a	a0	52	3b	a6	b3	29	a3	2f	64	72	8f	64	86	88	98	16	d4	a4	5c	cc	56	b5	b6	
50	53	d1	00	ed	20	fc	b1	5b	6a	cb	ba	39	4a	4c	58	cf	6c	70	40	50	fd	ed	b9	da	5e	15	46	57	a7	8d	
60	d0	ef	aa	fb	43	4d	33	05	46	19	02	7f	50	3c	9f	a0	90	d8	ab	00	0c	bc	d3	0a	ff	e4	58	05	b8	b3	
70	51	a3	a0	8f	92	94	38	f5	bc	b6	da	21	10	f0	63	42	70	4b	2c	1e	0f	ca	3f	0f	02	c1	af	bd	02	01	
80	cd	dc	13	ac	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73	80	3a	91	11	41	41	67	dc	aa	97	02	cf	ea	8b	
90	60	81	4f	cf	22	2a	90	88	46	ee	b8	14	de	5e	0b	db	90	96	ac	74	22	e7	ad	35	85	e2	37	a8	1c	75	
a0	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79	a0	47	f1	1a	71	14	29	c5	89	6f	b7	62	0e	aa	
b0	a7	c8	37	6d	8d	d5	4a	a9	6c	56	1a	aa	65	7a	aa	08	b0	f6	56	30	4b	c5	d2	79	20	9a	db	c0	1e	78	
c0	ba	78	25	2a	1c	a6	b4	c5	a8	d8	74	1f	4b	bd	8b	8a	c0	1f	dd	a0	33	88	07	c7	31	b1	12	10	59	27	
d0	70	3e	b5	66	48	03	6e	0e	61	35	57	b9	86	c1	1d	9e	d0	60	51	7f	a9	19	b5	4a	0d	2d	05	7a	9f	93	
e0	e1	88	58	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	d9	e0	a0	3b	d4	ae	2a	f5	b0	c8	bb	3c	83	03	53	
f0	8c	a1	09	08	b1	a5	e2	68	41	99	2d	0f	b0	54	b0	16	f0	1b	2b	04	7a	7b	05	26	e1	69	14	63	55	21	

(a) AES S-box

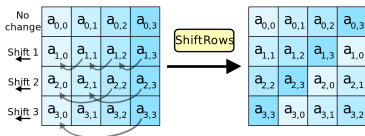
(b) AES Inverse S-box

SubBytes procedūroje, kiekvienas būsenos baitas yra pakeičiamas kitu. Baitų keitimui yra naudojama keitinių lentelė (angl. *substitution box* arba *S-box*), atvirkštinei procedūrai - atvirkštinė keitinių lentelė (angl. *inverse S-box*).

Šios operacijos tikslas - pakeisti baitus.

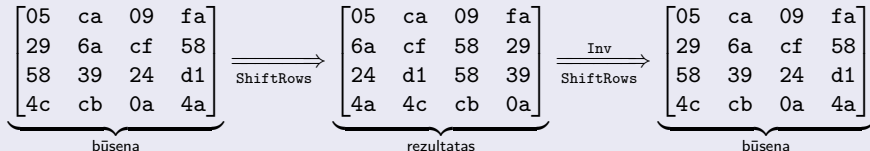


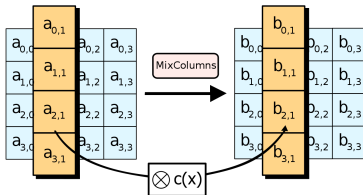
ShiftRows



ShiftRows procedūroje eilučių elementai paslenkami j kairē. Pirmā eilutē - nekeičiama, otra eilutē j kairē par 1, trešā par 2, ceturta par 3. Atvirkstīnei procedūrai daromi analogiski veiksmi j dešīnē.

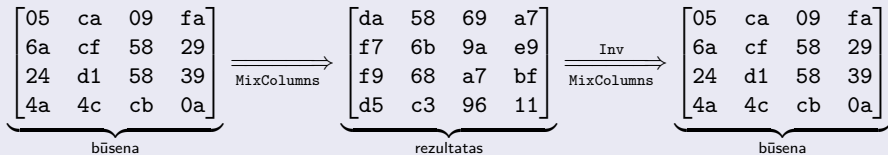
Šīs operācijas tikslas - sumaišyti baitus.





MixColumns procedūroje kiekvienas būsenos stulpelis yra transformuojamas, jį padauginant iš apgręžiamos matricos.

Šios operacijos tikslas - sumaišyti ir pakeisti baitus.



MixColumns operacijos i -tojo stulpelio daugyba atrodo taip:

$$\underbrace{\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}}_{\text{MixColumns matrica}} \underbrace{\begin{bmatrix} a_{0,i} \\ a_{1,i} \\ a_{2,i} \\ a_{3,i} \end{bmatrix}}_{\text{senas būsenos stulpelis}} = \underbrace{\begin{bmatrix} b_{0,i} \\ b_{1,i} \\ b_{2,i} \\ b_{3,i} \end{bmatrix}}_{\text{naujas būsenos stulpelis}}$$

Matricų daugyba atliekama panašiai kaip ir su paprastomis matricomis. Visi daugybos veiksmai yra keičiami $GF(2^8)$ daugyba (žymima $\bullet$), o visi sudėties veiksmai keičiami XOR operacijomis. Praktikoje, $\bullet$ veiksmams yra naudojamos daugybos lentelės.

Tada stulpelių reikšmės galima aprašyti taip:

$$\begin{aligned}
 b_{0,i} &= (02 \bullet a_{0,i}) \oplus (03 \bullet a_{1,i}) \oplus (01 \bullet a_{2,i}) \oplus (01 \bullet a_{3,i}) \\
 b_{1,i} &= (01 \bullet a_{0,i}) \oplus (02 \bullet a_{1,i}) \oplus (03 \bullet a_{2,i}) \oplus (01 \bullet a_{3,i}) \\
 b_{2,i} &= (01 \bullet a_{0,i}) \oplus (01 \bullet a_{1,i}) \oplus (02 \bullet a_{2,i}) \oplus (03 \bullet a_{3,i}) \\
 b_{3,i} &= (03 \bullet a_{0,i}) \oplus (01 \bullet a_{1,i}) \oplus (01 \bullet a_{2,i}) \oplus (02 \bullet a_{3,i})
 \end{aligned}$$

Tarkime, norime sudauginti $f_{2_{16}} \bullet 3_{16}$. Abu dauginamieji išreiškiami polinomais mod2:

$$f_{2_{16}} = 11110010_2 = x^7 + x^6 + x^5 + x^4 + x, \quad 3_{16} = 11_2 = x + 1$$

Abu polinamai sudauginami mod2:

$$\begin{aligned} (x^7 + x^6 + x^5 + x^4 + x)(x + 1) &= \\ &= x^8 + x^7 + x^6 + x^5 + x^2 + x^7 + x^6 + x^5 + x^4 + x \\ &= x^8 + 2x^7 + 2x^6 + 2x^5 + x^4 + x^2 + x \\ &= x^8 + x^4 + x^2 + x \end{aligned}$$

Gautą polinomą užrašome dvejetainė išraiška 100010110_2 . Kadangi skaičių sudaro daugiau nei 8 bitai, t.y. reikšmė didesnė, nei didžiausia galima baido reikšmė ff_{16} , gautą rezultatą redukuojame moduliu $x^8 + x^4 + x^3 + x + 1 = 100011011_2$

$$\begin{array}{r} 100010110 \\ \oplus \underline{100011011} \\ \hline 000001101 \end{array}$$

$$f_{2_{16}} \bullet 3_{16} = 000001101_2 = d_{16} = 13_{10}$$

InvMixColumns operacijos i -tojo stulpelio daugyba atrodo taip:

$$\underbrace{\begin{bmatrix} 0e & 0b & 0d & 09 \\ 09 & 0e & 0b & 0d \\ 0d & 09 & 0e & 0b \\ 0b & 0d & 09 & 0e \end{bmatrix}}_{\text{InvMixColumns matrica}} \underbrace{\begin{bmatrix} a_{0,i} \\ a_{1,i} \\ a_{2,i} \\ a_{3,i} \end{bmatrix}}_{\text{senas būsenos stulpelis}} = \underbrace{\begin{bmatrix} b_{0,i} \\ b_{1,i} \\ b_{2,i} \\ b_{3,i} \end{bmatrix}}_{\text{naujas būsenos stulpelis}}$$

Tada stulpelių reikšmės galima aprašyti taip:

$$b_{0,i} = (0e \bullet a_{0,i}) \oplus (0b \bullet a_{1,i}) \oplus (0d \bullet a_{2,i}) \oplus (09 \bullet a_{3,i})$$

$$b_{1,i} = (09 \bullet a_{0,i}) \oplus (0e \bullet a_{1,i}) \oplus (0b \bullet a_{2,i}) \oplus (0d \bullet a_{3,i})$$

$$b_{2,i} = (0d \bullet a_{0,i}) \oplus (09 \bullet a_{1,i}) \oplus (0e \bullet a_{2,i}) \oplus (0b \bullet a_{3,i})$$

$$b_{3,i} = (0b \bullet a_{0,i}) \oplus (0d \bullet a_{1,i}) \oplus (09 \bullet a_{2,i}) \oplus (0e \bullet a_{3,i})$$